

数据结构

Ch2 线性表

计算机学院（国家示范性软件学院）

第2章 线性表

- 2.1 线性表
- 2.2 线性表的顺序存储与实现
- 2.3 线性表的链式存储与实现
- 2.4 顺序表和链表的比较
- 2.5 单链表的应用：多项式及其运算
- 2.6 本章知识点小结

线性结构

- 何谓线性结构？
 - 线性结构是一个数据元素的有序（次序）集合。

例1、26个英文字母组成的字母表
(A, B, C, ..., Z)



例2、一副扑克的点数
(2, 3, 4, ..., J, Q, K, A)



线性结构

- 四个特征：
 - 集合中必存在**唯一**的一个“**第一元素**”
 - 集合中必存在**唯一**的一个“**最后元素**”
 - **除最后元素之外**，其它数据元素均有**唯一**的“**后继**”
 - **除第一元素之外**，其它数据元素均有**唯一**的“**前驱**”
- 常见的线性结构：
 - 线性表、栈、队列、循环队列、数组、串

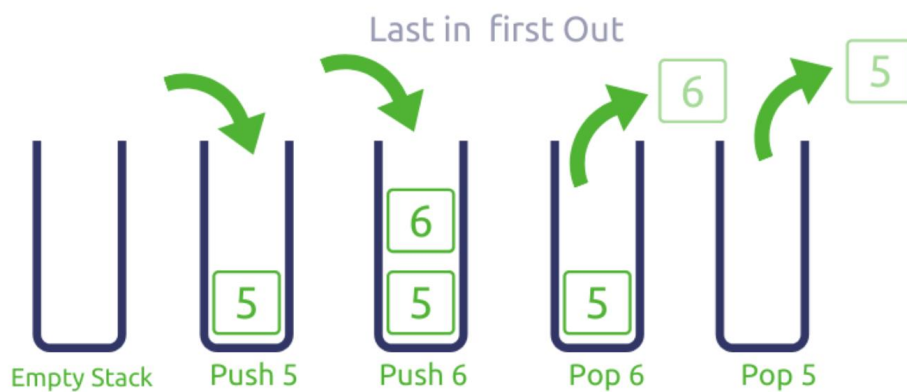
线性结构

例1：日常生活中看到的学生花名册、学生的成绩单、通讯录、单位的职工工资表以及图书馆的图书目录等等，这些表单具有一个共同的特点，都是由一行行结构相同的数据构成。对这些表单经常进行的操作是修改、查找、插入和删除。

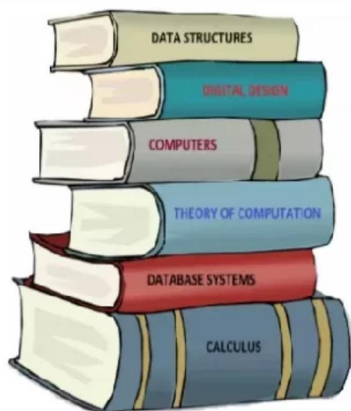
学号	姓名	数学	物理	英语
880001	丁一	90	85	95
880002	马二	80	85	90
880003	张三	95	90	99
880004	李四	70	84	86
.....				

线性结构

例2：元素之间的逻辑关系是线性结构，但处理顺序需要满足**后进先出**的原则。



Ball Stack



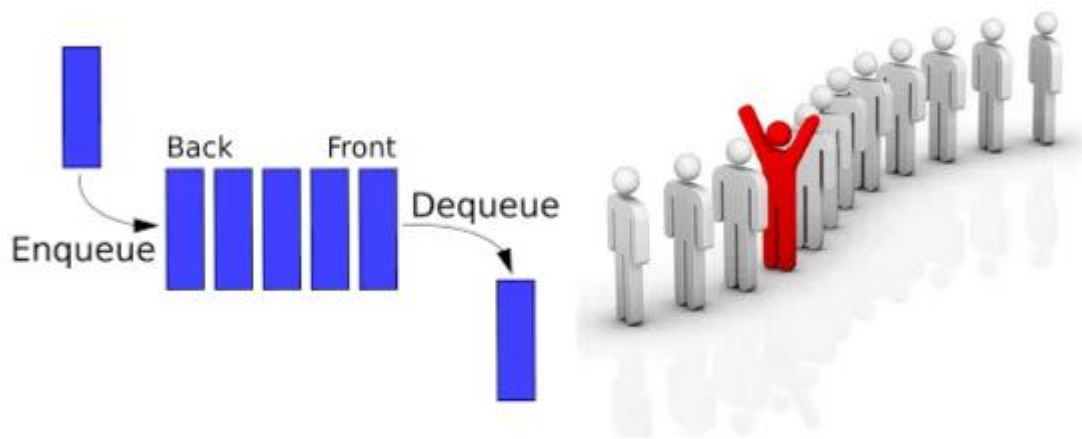
Book Stack



CD Stack

线性结构

例3：日常生活中排队购物、汽车进站出站、到银行办理业务等事务处理过程，如果用计算机模拟，一般情况是一个需要遵循先来先服务（**先进先出**）的处理原则的线性结构。



线性结构

例4：文本编辑软件对文本的处理，可以认为是对一个大字符串进行建立、插入、删除、取子串、字符串匹配等操作，而字符串的逻辑结构是**以字符为元素**的线性结构。

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

A	A	B	A						A	A	B	A			
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												A	A	B	A

Pattern Found at 0, 9 and 12

线性表定义

线性表是最简单、最基本、也是最常用的线性结构

具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列，
通常记为：

$$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$$

其中 n 为表长， $n = 0$ 时称为空表。

线性表的ADT定义

- **ADT List {**
- 数据对象 (D) :
 - $D = \{a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0\}$
- 数据关系 (S) :
 - $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,\dots,n \}$
 - 序偶, 即有序对, 具有确定的次序两个元素的集合。

线性表的ADT定义

- 基本操作 (P) :
 - (1) 线性表初始化: InitList(&L)
 - (2) 销毁线性表: DestroyList(&L)
 - (3) 将线性表置空: ClearList(&L)
 - (4) 判断线性表是否为空: ListEmpty(L)
 - (5) 求线性表的长度: ListLength (L)
 - (6) 取表元: GetElem(L,i,&e)

线性表的ADT定义

- 基本操作:

- (7)按值查找: `LocateElem(L,e,compare())`
 - 返回满足条件的数据元素的位序, 如果不存在返回0
- (8)前驱: `PriorElem(L,cur_e, &pre_e)`
- (9)后继: `NextElem(L,cur_e,&next_e)`
- (10)插入操作: `ListInsert (&L,i,e)`
- (11) 删除操作: `ListDelete (&L,i,&e)`
- (12) 遍历操作: `ListTraverse (L,visit())`
 - 依次对L的每个数据元素调用函数`visit()`

例2-1

- 现假设以线性表 LA 和 LB 分别表示集合 A 和 B，求两个集合的并集，使 $A = A \cup B$ 。

上述集合求并的问题演绎为：扩大线性表 LA，**将存在于线性表 LB 中而不存在于线性表 LA 中的数据元素插入到线性表 LA 中去。**

例2-1

具体操作步骤为:

1. 从线性表 LB 中取出一个数据元素;
 2. 依值在线性表 LA 中进行查询;
 3. 若不存在, 则将它插入到 LA 中。
- 重复上述三步直至 处理完LB中所有的元素。

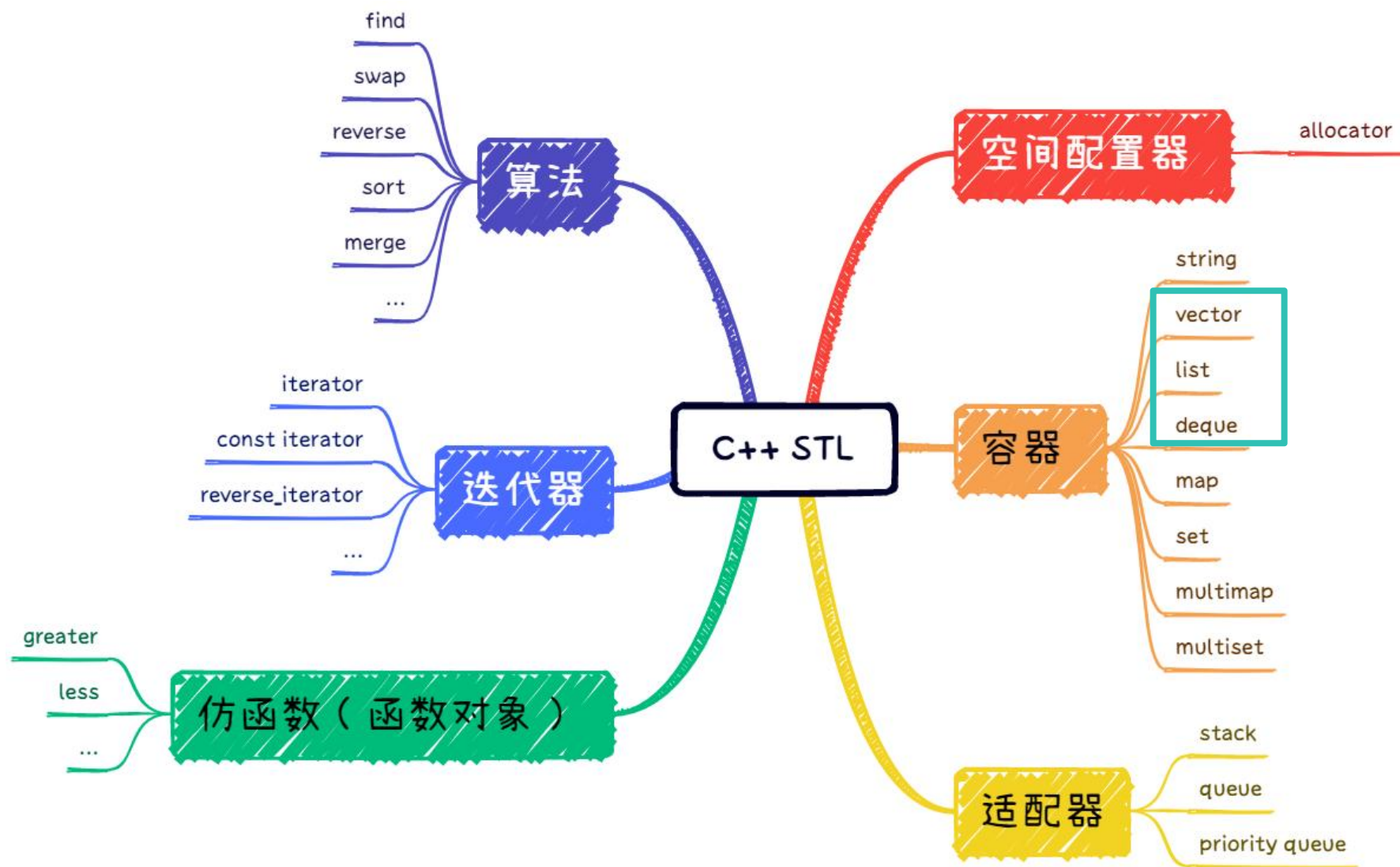
对应以下基本操作

1. ListLength(Lb)
2. GetElem(Lb, i, &e)
3. LocateElem(La, e);
4. ListInsert(La, n+1,e)

算法伪代码:

```
void union(List &La, List Lb){  
    La_len = ListLength(La); //求线性表La的长度  
    Lb_len = ListLength(Lb); //求线性表Lb的长度  
    for(i=1; i<=Lb_len; i++){ //遍历线性表Lb  
        GetElem(Lb,i,e) //取Lb中第i个数据元素赋给e  
        if(!LocateElem(La,e,equal)){//如果e在La中不存在  
            ListInsert(La, ++La_len,e); //将e插入到La的末位  
        }  
    }  
}
```

C++ STL 标准模板库 (Standard Template Library)



C++ STL 标准模板库 (Standard Template Library)

STL顺序容器<--->线性结构

std::vector: 动态数组, 支持快速随机访问。

std::list: 链表, 支持快速插入和删除, 但不支持随机访问。

std::deque: 双端队列, 支持快速插入和删除。

```
// 创建一个向量存储 int
vector<int> vec;
int i;

// 显示 vec 的原始大小
cout << "vector size = " << vec.size() << endl;

// 推入 5 个值到向量中
for(i = 0; i < 5; i++){
    vec.push_back(i);
}
```

C++ STL 标准模板库 (Standard Template Library)

SGI 版本STL源代码

https://www.rrsd.com/software_development/stl/stl/download.html



Hosted by

Robert Ramey
Software Development

Download STL source code

Last updated: 8-June-00

The STL was developed on SGI MIPSpro™ C++ 7.0, 7.1, 7.2, and 7.2.1. If you are using the 7.0 compiler, you must compile using either the `-n32` or the `-64` flag; if you are using 7.1 or later, you may use `-o32`, `-n32`, or `-64`. The STL has also been tested on Microsoft Visual C++ 5.0, on g++ 2.8.1, and on recent [egcs](#) snapshots. All of the STL except for the `<string>`, `<bitset>`, and `<valarray>` headers has also been tested on Borland 5.02.

It should not be difficult to use the STL with other compilers that have good support for templates, especially compilers that use the [EDG](#) front end.

This distribution of the STL consists entirely of header files: there is no need to link to any library files. You can view or download a header file individually, or you can download all of them as a tar or zip file. (Note that v3 has many more header files than v1 and v2 did, because it provides both old-style and new-style header names. Many of the files in v3 are very short, and do little other than forwarding.)

View individual files

algo.h	hash_map.h	numeric	stdexcept	stl_heap.h	stl_slist.h
algorith	hash_set	pair.h	stl_algo.h	stl_iterator.h	stl_stack.h
alloc.h	hash_set.h	pthread_alloc	stl_algorith	stl_iterator_base.h	stl_string_fwd.h
	hashtable.h	pthread_alloc.h	stl_alloc.h	stl_list.h	stl_tempbuf.h

C++ STL 标准模板库 (Standard Template Library)

<https://www.geeksforgeeks.org/vector-in-cpp-stl/>

迭代器 Iterators

begin() – Returns an iterator pointing to the first element in the vector

end() – Returns an iterator pointing to the theoretical element that follows the last element in the vector

rbegin() – Returns a reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element

rend() – Returns a reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end)

cbegin() – Returns a constant iterator pointing to the first element in the vector.

chend() – Returns a constant iterator pointing to the theoretical element that follows the last element in the vector.

crbegin() – Returns a constant reverse iterator pointing to the last element in the vector (reverse beginning). It moves from last to first element

crend() – Returns a constant reverse iterator pointing to the theoretical element preceding the first element in the vector (considered as reverse end)

C `#include <iostream>`
`#include <vector>`

y)

<https://>

```
using namespace std;

int main()
{
    vector<int> g1;

    for (int i = 1; i <= 5; i++)
        g1.push_back(i);

    cout << "Output of begin and end: ";
    for (auto i = g1.begin(); i != g1.end(); ++i)
        cout << *i << " ";

    cout << "\nOutput of cbegin and cend: ";
    for (auto i = g1.cbegin(); i != g1.cend(); ++i)
        cout << *i << " ";

    cout << "\nOutput of rbegin and rend: ";
    for (auto ir = g1.rbegin(); ir != g1.rend(); ++ir)
        cout << *ir << " ";

    cout << "\nOutput of crbegin and crend : ";
    for (auto ir = g1.crbegin(); ir != g1.crend(); ++ir)
        cout << *ir << " ";

    return 0;
}
```

C++ STL 标准模板库 (Standard Template Library)

<https://www.geeksforgeeks.org/vector-in-cpp-stl/>

容量Capacity

size() – Returns the number of elements in the vector.

max_size() – Returns the maximum number of elements that the vector can hold.

capacity() – Returns the size of the storage space currently allocated to the vector expressed as number of elements.

resize(n) – Resizes the container so that it contains ‘n’ elements.

empty() – Returns whether the container is empty.

shrink_to_fit() – Reduces the capacity of the container to fit its size and destroys all elements beyond the capacity.

reserve() – Requests that the vector capacity be at least enough to contain n elements.

C++

```
int main()
```

```
{
```

```
    vector<int> g1;
```

```
    for (int i = 1; i <= 5; i++)  
        g1.push_back(i);
```

```
    cout << "Size : " << g1.size();  
    cout << "\nCapacity : " << g1.capacity();  
    cout << "\nMax_Size : " << g1.max_size();
```

```
    // resizes the vector size to 4  
    g1.resize(4);
```

```
    // prints the vector size after resize()  
    cout << "\nSize : " << g1.size();
```

```
    // checks if the vector is empty or not
```

```
    if (g1.empty() == false)  
        cout << "\nVector is not empty";
```

```
    else  
        cout << "\nVector is empty";
```

```
    // Shrinks the vector
```

```
    g1.shrink_to_fit();  
    cout << "\nVector elements are: ";  
    for (auto it = g1.begin(); it != g1.end(); it++)  
        cout << *it << " ";
```

```
    return 0;
```

```
}
```

ary)

<https://www>

C++ STL 标准模板库 (Standard Template Library)

<https://www.geeksforgeeks.org/vector-in-cpp-stl/>

Element access

- reference operator [g]** – Returns a reference to the element at position ‘g’ in the vector
- at(g)** – Returns a reference to the element at position ‘g’ in the vector
- front()** – Returns a reference to the first element in the vector
- back()** – Returns a reference to the last element in the vector
- data()** – Returns a direct pointer to the memory array used internally by the vector to store its owned elements.

C++ STL 标准模板库 (Standard Template Library)

<https://www.geeksforgeeks.org/vector-in-cpp-stl/>

Element access

```
int main()
{
    vector<int> g1;

    for (int i = 1; i <= 10; i++)
        g1.push_back(i * 10);

    cout << "\nReference operator [g] : g1[2] = " << g1[2];

    cout << "\nat : g1.at(4) = " << g1.at(4);

    cout << "\nfront() : g1.front() = " << g1.front();

    cout << "\nback() : g1.back() = " << g1.back();

    // pointer to the first element
    int* pos = g1.data();

    cout << "\nThe first element is " << *pos;
    return 0;
}
```

C++ STL 标准模板库 (Standard Template Library)

<https://www.geeksforgeeks.org/vector-in-cpp-stl/>

Modifiers

assign() – It assigns new value to the vector elements by replacing old ones

push_back() – It push the elements into a vector from the back

pop_back() – It is used to pop or remove elements from a vector from the back.

insert() – It inserts new elements before the element at the specified position

erase() – It is used to remove elements from a container from the specified position or range.

swap() – It is used to swap the contents of one vector with another vector of same type.

Sizes may differ.

clear() – It is used to remove all the elements of the vector container

emplace() – It extends the container by inserting new element at position

emplace_back() – It is used to insert a new element into the vector container, the new element is added to the end of the vector

2.2 线性表的顺序存储与实现

- “用一组地址连续的存储单元依次存放线性表中的数据元素”，即以“存储位置相邻”表示“位序相继的两个数据元素之间的前驱和后继的关系(有序对 $\langle a_{i-1}, a_i \rangle$)”，并以表中第一个元素的存储位置作为线性表的起始地址，称作线性表的基地址。



2.2 线性表的顺序存储与实现

- 顺序存储可以按序号随机访问每一个数据元素:

设 a_1 的存储地址为 $\text{Loc}(a_1)$, 每个数据元素占 d 个存储地址, 则第 i 个数据元素的地址为:

$$\text{Loc}(a_i) = \text{Loc}(a_1) + (i-1) * d \quad 1 \leq i \leq n$$

可以 用一维数组来示实现顺序表的数据存储区域
可以是:

```
typedef struct {  
    ElemType *elem; // 存储空间基址  
    int length;      // 当前长度  
    int listsize;    // 允许的最大存储容量  
} SqList;           // 俗称 顺序表
```

2.2 线性表的顺序存储与实现

- 几个主要操作的实现
 - 1、初始化操作
 - 2、取表元操作
 - 3、插入元素操作
 - 4、删除元素操作
 - 5、元素定位操作

2.2 线性表的顺序存储与实现

1、顺序表的初始化

```
bool InitList_Sq(SqList &L)
{
    // 构造一个空的线性表 L
    L.elem =
    (ElemType*)malloc(LIST_INIT_SIZE*sizeof(ElemType));
    if (!L.elem)    exit(OVERFLOW);    // 存储分配失败
    L.length = 0;    // 顺序表的初始长度为0
    L.listsize = LIST_INIT_SIZE; // 初始存储容量
    return OK;
} // InitList_Sq
```

此算法的时间复杂度为 $O(1)$ 。

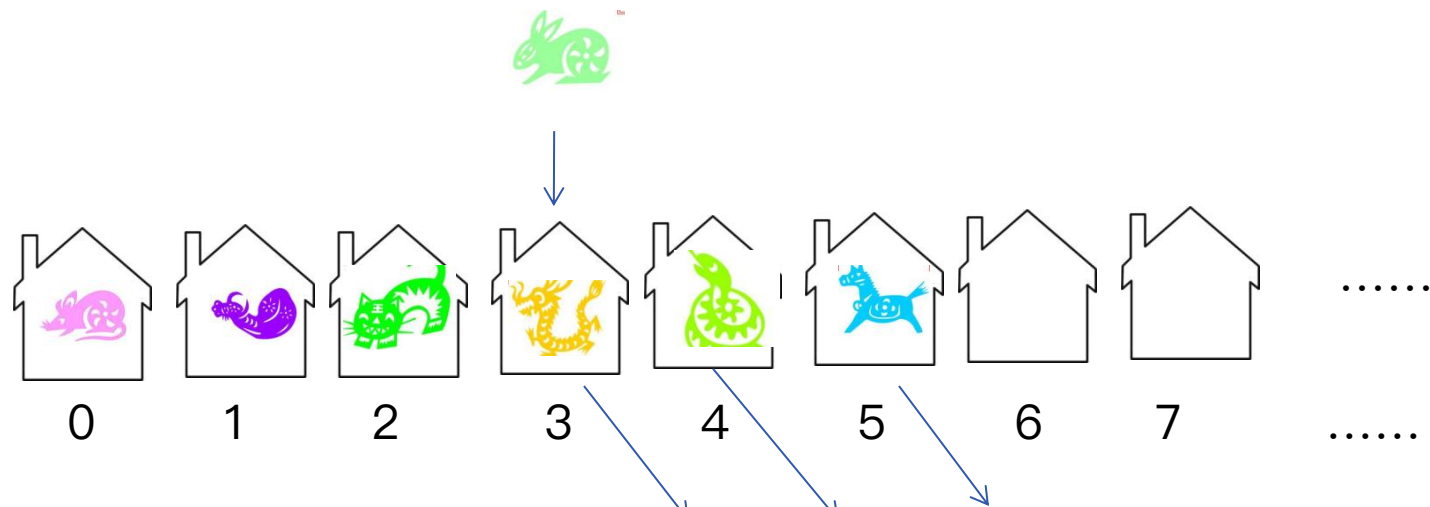
2.2 线性表的顺序存储与实现

2、按位序取表元操作

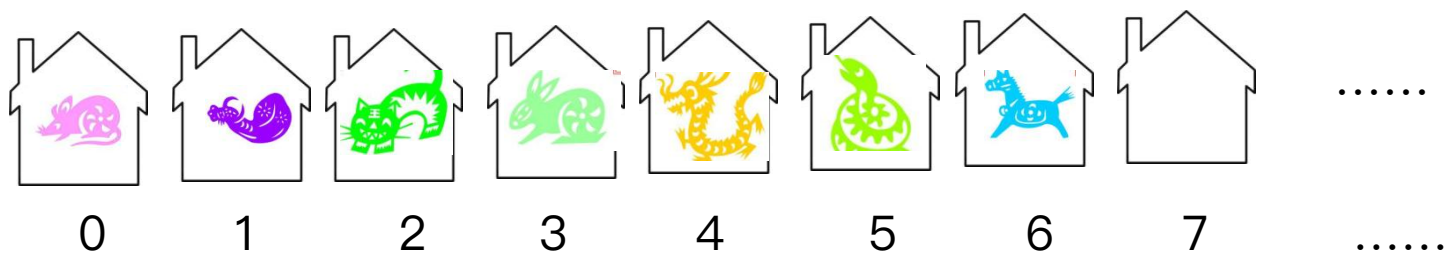
```
bool GetElem_Sq(L,i,&e)
{
    if(i<1 || i> L.length) return ERROR;//i值不合法
    e = L.elem[i-1];
    return OK;
}
```

3、顺序表的插入，指在表的第 i 个位置上插入一个值为 x 的新元素

插入前



插入后



2.2 线性表的顺序存储与实现

- 插入运算思路:

将 $a_1 \sim a_n$ 顺序向后移动, 为新元素让出位置

将 x 置入空出的第 i 个位置

修改 $L.length$ (相当于修改表长)

- 思路代码化:

if ($i < 1 \parallel i > L.length + 1$)

超出范围

if ($L.length \geq L.listsize$)

满, 返回false, 或增加分配空间

从 a_n 到 a_i 依次后移一个位置

将 x 置入空出的第 i 个位置

修改 $L.length$

2.2 线性表的顺序存储与实现

- **bool** ListInsert_Sq(SqList &L, **int** i, ElemType e)
 { **if** (i < 1 || i > L.length+1) **return ERROR** ; // 插入位置不合法
 if (L.length >= L.listsize) **return ERROR**; // 当前存储空间已满, 无法插入

 for (j=L.length; j>=i; j--) { // 从最后一个到插入位置
 L.elem[j] = L.elem[j-1]; } // 插入位置及之后元素右移
 L.elem[i-1] = e; // 插入 e
 L.length++; // 表长增1
 return TRUE;
 } // ListInsert
- 此处可优化

2.2 线性表的顺序存储与实现

- **bool** ListInsert_Sq(SqList &L, **int** i, ElemType e)
 { **if** (i < 1 || i > L.length+1) **return ERROR** ; // 插入位置不合法
 if (L.length >= L.listsize) **return ERROR**; // 当前存储空间已满, 无法插入
 q = &(L.elem[i-1]) // q为插入位置
 for(p = &(L.elem[L.length-1]); p >= q; --p){
 *(p+1) = *p; } // 插入位置及之后元素右移
 L.elem[i-1] = e; // 插入 e
 L.length++; // 表长增1
 return TRUE;
 } // ListInsert

2.2 线性表的顺序存储与实现

本算法中注意以下问题:

① 顺序表中数据区域有MAXSIZE个存储单元, 所以在向顺序表中做插入时先检查表空间是否满了, 在表满的情况下除非扩展空间否则不能再做插入。

② 要检验插入位置的有效性, 这里 插入位置*i*的有效范围是:
 $1 \leq i \leq n+1$, 其中 *n* 为原表长。

③ 注意数据的移动方向。

④ 表长的修改。

2.2 线性表的顺序存储与实现

插入算法的**时间性能分析**：

平均移动数据元素的次数：

$$E_{in} = \sum_{i=1}^{n+1} p_i (n - i + 1)$$

设： $P_i = 1 / (n+1)$ ，即为等概率情况，则：

$$E_{in} = \sum_{i=1}^{n+1} p_i (n - i + 1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{n}{2}$$

这说明：在顺序表上做插入操作需移动表中一半的数据元素。显然时间复杂度为 $O(n)$ 。

2.2 线性表的顺序存储与实现

- 4、删除元素运算

- “删除元素”使线性表的逻辑结构发生什么变化?

假设删除线性表中第 i 个元素，使得线性表

删除前: $(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$

删除后: $(a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$

(1) 改变了表中元素之间的关系，使

$\langle a_{i-1}, a_i \rangle$ 和 $\langle a_i, a_{i+1} \rangle$ 改变为 $\langle a_{i-1}, a_{i+1} \rangle$

(2) 表长减1

2.2 线性表的顺序存储与实现

- 删除元素运算基本思路:
 - 1) 将 $a_{i+1} \sim a_n$ 顺序向前移动
 - 2) 修改`L.length`
 - 思路代码化:
 - `if (i<1||i>n)` //检查空表及删除位置的合法性
 - 不存在
 - 将 $a_{i+1} \sim a_n$ 顺序向前移动
- 修改`L.length`

2.2 线性表的顺序存储与实现

- **bool** ListDelete_Sq(SqList &L, **int** i, ElemType &e)
 { **if** ((i < 1) || (i > L.length)) **return FALSE** ; // 删除位置不合法
 e = L.elem[i-1]
 for (j = i+1; j <= L.length; ++j) { // 从i后面一个开始一直到最后
 L.elem[j-2] = L.elem[j-1]; } // 被删除元素之后的元素左移
 L.length--; // 表长减1
 return TRUE;
 } // ListDelete

2.2 线性表的顺序存储与实现

本算法中注意以下问题：

- ① 删除第 i 个元素， i 的取值为 $1 \leq i \leq n$ ，否则第 i 个元素不存在，因此，要检查删除位置的有效性。
- ② 当表空时不能做删除，因表空时 $L.length$ 的值为 0，条件 $(i < 1 || i > L.length)$ 也包括了对表空的检查。
- ③ 删除 a_i 之后，该数据已不存在，如果需要，先取出，再做删除。
- ④ 修改表长。

2.2 线性表的顺序存储与实现

删除算法的时间性能分析：

平均移动数据元素的次数：

$$E_{\text{删}} = \sum_{i=1}^n p_i (n - i)$$

在等概率情况下， $p_i = 1/n$ ，则：

$$E_{\text{删}} = \sum_{i=1}^n p_i (n - i) = \frac{1}{n} \sum_{i=1}^{n+1} (n - i) = \frac{n - 1}{2}$$

结论：顺序表上作删除运算时大约需要移动表中一半的元素。算法的时间复杂度为 $O(n)$ 。

2.2 线性表的顺序存储与实现

5、元素定位操作

在顺序表中“查询”是否存在一个和给定值满足判定条件的元素的最简单的办法是，依次取出结构中的每个元素和给定值进行比较。

2.2 线性表的顺序存储与实现

- **int** LocateElem_Sq(SqList L, ElemType e)
 {// 在顺序表L中查找第1个值与 e 满足相等条件的元素
 // 若找到，则返回其在 L 中的位序，否则返回0。
 i = 1; // i 的初值为第1元素的位序
 p = L.elem; // p 的初值为第1元素的存储位置
 while (i <= L.length && !equal(*p++,e))
 ++i; // 依次进行判定
 if (i <= L.length)
 return i; // 找到满足判定条件的数据元素为第 i 个元素
 else
 return 0; // 该线性表中不存在满足判定的数据元素
 } // LocateElem

2.2 线性表的顺序存储与实现

按值查找算法的时间性能分析：

本算法的主要运算是给定值 x 与表中元素的比较。

显然比较的次数与 x 在表中的位置有关，也与表长有关。

- 当 $a_1=x$ 时，比较一次成功。
- 当 $a_n=x$ 时，比较 n 次成功。

等概率情况下，查找成功的平均比较次数为：

$$E_{lo} = \frac{1}{n} \sum_{i=1}^n i = \frac{n+1}{2}$$

即：平均比较次数为 $(n+1) / 2$ ，时间性能为 $O(n)$ 。

2.2 线性表的顺序存储与实现

- 顺序表应用举例

应用1: 将顺序表($a_1, a_2, \dots, a_n$)重新排列为以 a_1 为界的两部分: a_1 前面的值均比 a_1 小, a_1 后面的值都比 a_1 大。

25	30	20	60	10	35	15		
15	10	20	25	30	60	35		

- 基本思路:

从第二个元素开始到最后一个元素, 逐一向后扫描

- 1) 当前数据元素比 a_1 大时, 表明它已经在后面, 不必改变位置, 继续比较下一个
- 2) 当前数据元素比 a_1 小, 需要将它前面的元素都依次向后移动一个位置, 然后把它置于最前面。 (可以优化)

2.2 线性表的顺序存储与实现

//顺序表划分算法

```
void part(SqList &L) {  
    int i, j;  
    ElemType x, y;  
    x = L.elem[0]; //将基准数据元素置入x中  
    for (i = 2; i <= L.length; i++) { //逐一遍历其后元素  
        if (L.elem[i - 1] < x){ //如果当前元素小于基准元素,则把它移动到最前  
            y = L.elem[i - 1]; //将当前元素置于y中  
            for (j = i - 1; j >= 1; j--) //当前元素之前的元素依次后移  
                L.elem[j] = L.elem[j - 1];  
            L.elem[0] = y; //  
        }  
    }  
}
```

最坏情况下总的移动次数为：

$$\sum_{i=2}^n (i - 1 + 2) = \sum_{i=2}^n (i + 1) = \frac{(n - 1) * (n + 4)}{2}$$

即最坏情况下移动数据时间性能为 $O(n^2)$ 。

2.2 线性表的顺序存储与实现

- 有顺序表A和B，其元素均按从小到大的升序排列，编写一个算法将它们合并成一个顺序表C，要求C的元素也是从小到大的升序排列。

A	2	4	5	7	9	10
---	---	---	---	---	---	----

B	1	3	6	9	11	12
---	---	---	---	---	----	----

2.2 线性表的顺序存储与实现

//顺序表的合并

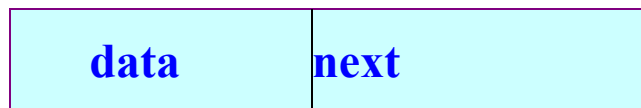
```
void merge(SqList A, SqList B, SqList &C){  
    int i, j, k;  
    i=0; j=0; k=0;  
    while(i<A.length && j<B.length)  
        //将A和B的当前元素较小者复制到C  
        if(A.elem[i]<B.elem[j])  
            C.elem[k++]=A.elem[i++];  
        else  
            C.elem[k++]=B.elem[j++];  
    while(i<A.length) //将A中的剩余元素复制到表C  
        C.elem[k++]=A.elem[i++];  
    while(j<B.length) //将B中的剩余元素复制到表C  
        C.elem[k++]=B.elem[j++];  
    C.length = A.length + B.length;  
}
```

算法的时间性能是 $O(m+n)$ ，其中 m 是A的表长， n 是B的表长。

2.3 线性表的链式存储与实现

- 单链表

- 为建立起数据元素之间的线性关系，对每个数据元素 a_i ，除了存放数据元素的自身的信息 a_i 之外，还需要和 a_i 一起存放其后继元素 a_{i+1} 所在的存储单元的地址，这两部分信息组成一个“结点”，结点的结构如图所示：



2.3 线性表的链式存储与实现

- 单链表



101



102



103



104



105



106



107



108

.....



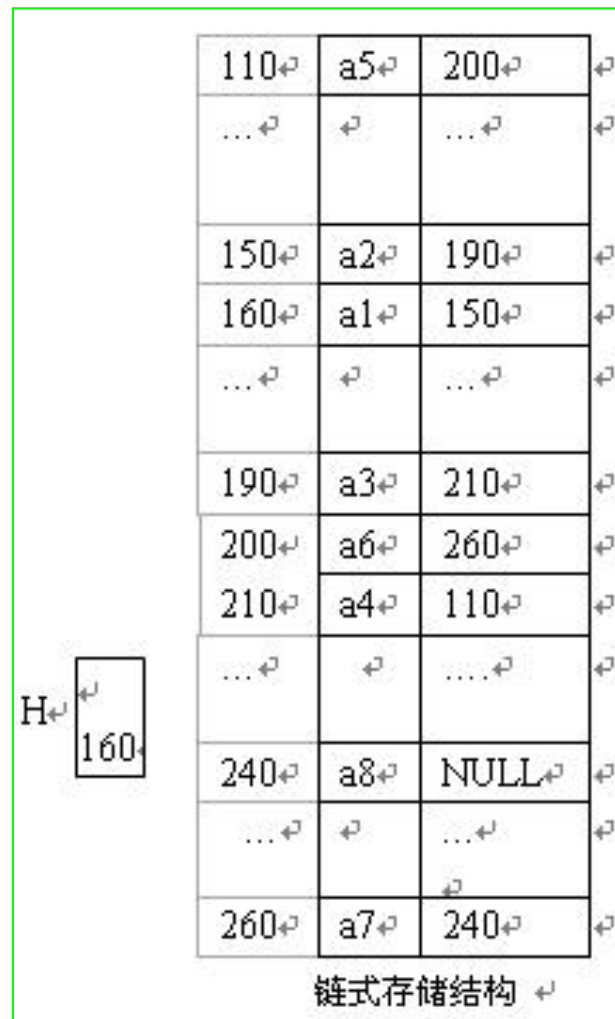
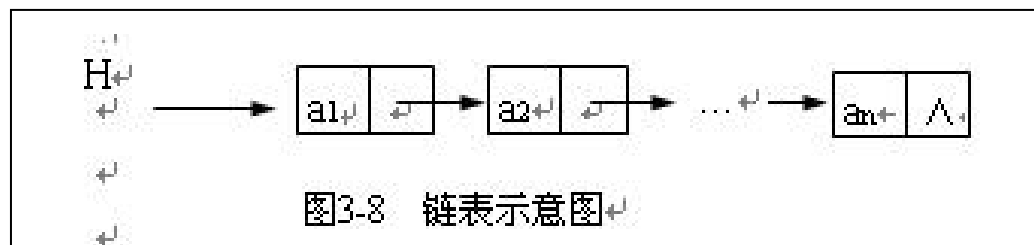
data

+

地址条107

next

2.3 线性表的链式存储与实现



2.3 线性表的链式存储与实现

结点定义如下：

```
typedef struct LNode
```

```
{ ElemType data;
```

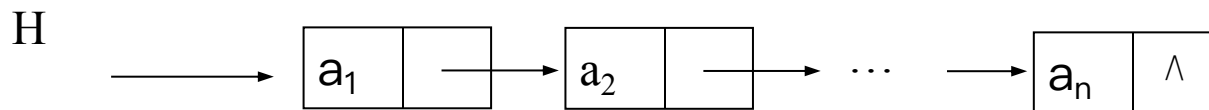
```
  struct LNode *next;
```

```
} LNode, *LinkList;
```

定义了结点类型

定义了指针类型

定义头指针变量： LinkList H;

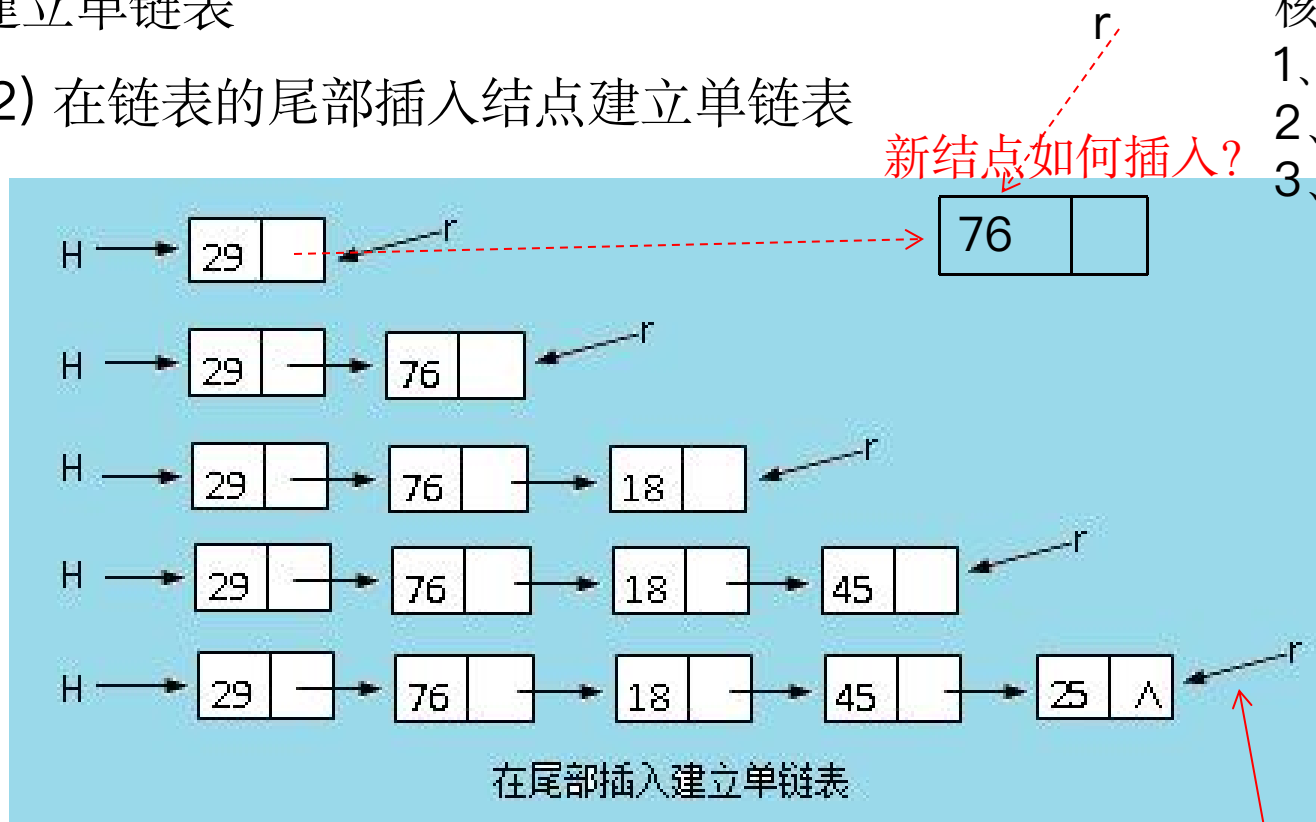


链表示意图

2.3 线性表的链式存储与实现

建立单链表

(2) 在链表的尾部插入结点建立单链表



核心操作:
1、创建新结点s
2、 $r \rightarrow \text{next} = s$
3、 $r = s$

在尾部插入结点需要**先找到尾部**

r 就是用来记录尾部在哪里的指针

2.3 线性表的链式存储与实现

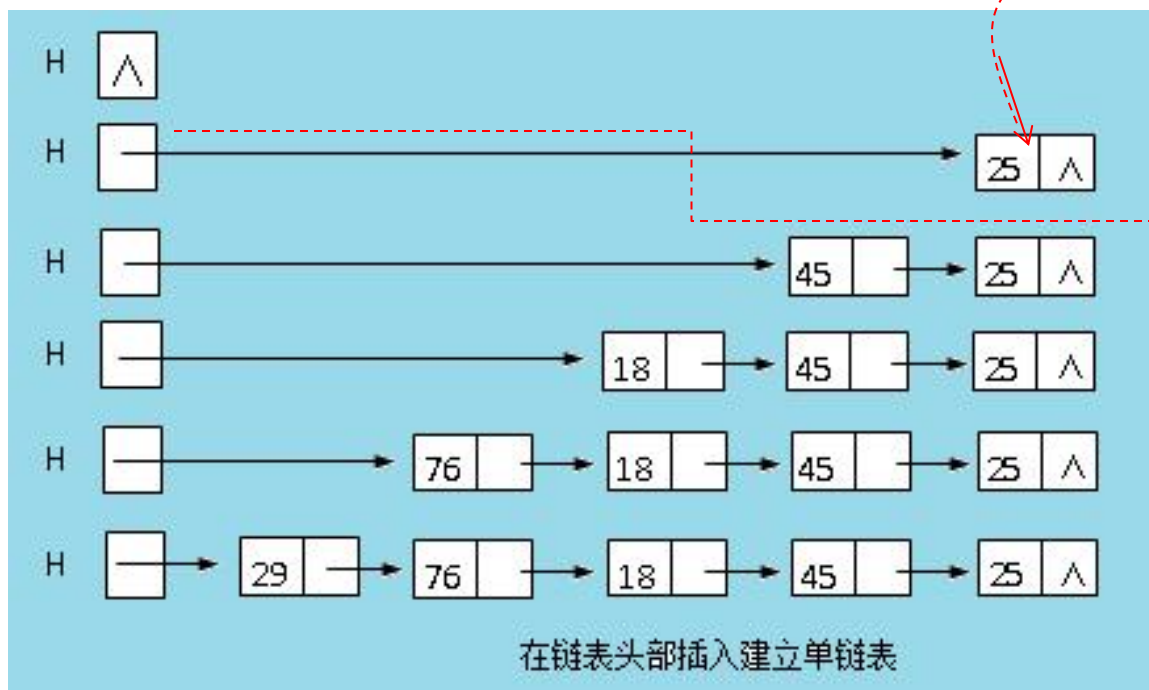
//在表尾插入，建立链表

```
LinkedList CreateList_T() {  
    LinkedList L = NULL;  
    LNode *s, *r = NULL;  
    int x;  
    std::cin >> x;  
    while (x != flag) { //flag 表示结束标志  
        s = new LNode;  
        s->data = x;  
        if (L == NULL) L = s; //第一个结点的处理  
        else r->next = s; //其它结点的处理  
        r = s;  
        std::cin >> x;  
    }  
    if (r != NULL) r->next = NULL; //将最后结点的指针域置空  
    return L;  
}
```

2.3 线性表的链式存储与实现

建立单链表

(1) 在链表的头部插入结点建立单链表



2.3 线性表的链式存储与实现

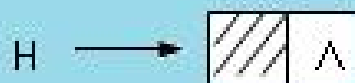
```
//在表头插入，建立链表
LinkedList CreateList_H() {
    LinkedList L = NULL;
    LNode *s;
    int x;
    std::cin >> x;
    while (x != flag) { //flag 表示结束标志
        s = new LNode;
        s->data = x;
        s->next = L; //新结点指向链表原来的第一个结点
        L = s; //L指向新结点
        std::cin >> x;
    }
    return L;
}
```

核心操作:

1. 创建一个新结点: `s=(Lnode *)malloc(sizeof(LNode))`
2. 新结点装入数据: `s->data=x`
3. 让s指向原来的第一个结点: `s->next =L`
4. 让头指针指向新创建的结点s: `L=s`

头结点问题的说明：

头结点的加入完全是为了运算的方便，它的数据域无定义，指针域中存放的是第一个数据结点的地址，空表时空。



(a) 带头结点的空单链表



(b) 带头结点的非空单链表

带头结点的单链表

3. 求表长

//设L是带头结点的单链表

```
int ListLength_1(LinkList L) {  
    //求带头结点的单链表表长  
    LNode *p = L; //p指向头结点  
    int j = 0; //计数器初始化为0  
    while (p->next) { //p->next不为空, 则计数器++  
        p = p->next;  
        j++;  
    }  
    return j;  
}
```

//设L是不带头结点的单链表

```
int ListLength_2(LinkList L) {  
    //求不带头结点的单链表表长  
    LNode *p = L; //非空情况下, p指向第一个结点  
    int j = 0; //计数器初始化为0  
    while (p) {  
        p = p->next;  
        j++;  
    }  
    return j;  
}
```

3.查找操作

(1)按序号查找 `GetElem(LinkList L, int i, ElemType &e)`

算法思路：从链表的第一个元素结点起，判断当前结点是否是第*i*个，若是，则返回该结点的指针，否则继续后一个，表结束为止。没有第*i*个结点时返回空指针。

```
} Status GetElem(LinkList L, int i, ElemType &e) {  
    //在带头结点的单链表L中根据序号i获取元素的值，用e返回  
    LNode *p = L->next; //初始化，p指向首元结点  
    int j = 1; //计数器j初值为1  
    while (p && j < i) { //顺链扫码，直至p为空或者p指向第i个元素  
        p = p->next; //p指向下一个结点  
        ++j; //计数器加1  
    }  
    if (!p || j > i) return ERROR; //未找到  
    e = p->data; //取第i个结点的数据域  
    return OK;  
}
```

(2)按值查找即定位 Locate_LinkList(L,x)

算法思路：从链表的第一个元素结点起，判断当前结点值是否等于x，若是，返回该结点的指针，否则继续后一个，表结束为止。找不到时返回空指针。

```
LNode *LocateElem(LinkList L, ElemType e){  
    //在带头结点的单链表中查找值为e的元素  
    LNode *p = L->next; //初始化，p指向首元结点  
    while(p && p->data!=e)//顺链扫描，直至p为空或p指向结点的数据域等于e  
    {  
        p = p->next;  
    }  
    return p; //查找成功返回e的结点地址，查找失败p为NULL  
}
```

4.插入操作

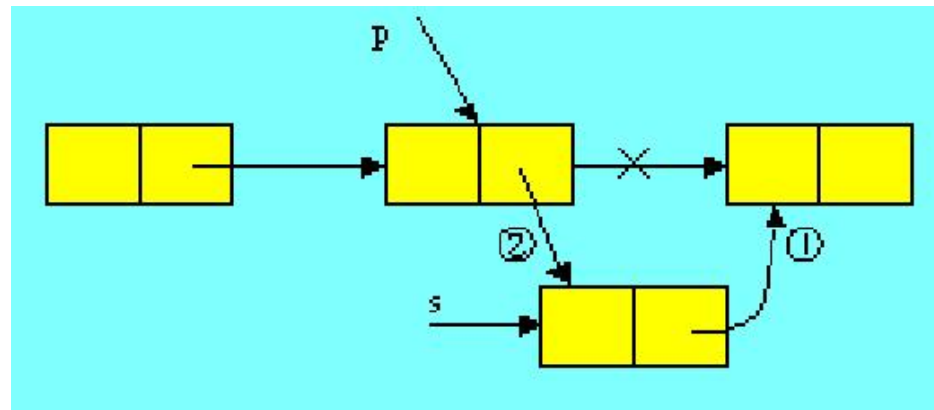
(1)后插结点：设p指向单链表中某结点，s指向待插入的值为x的新结点，将*s插入到*p的后面。

操作如下：

① $s \rightarrow \text{next} = p \rightarrow \text{next};$

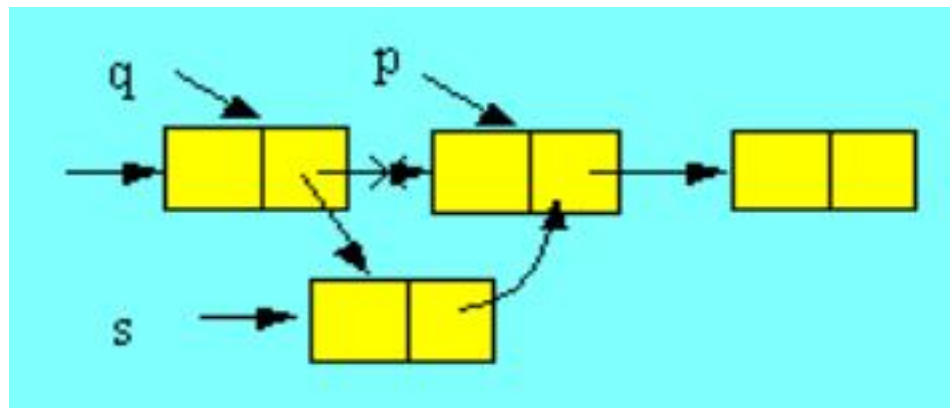
② $p \rightarrow \text{next} = s;$

注意：两个指针的操作顺序不能交换。



(2) 前插结点：设 p 指向链表中某结点， s 指向待插入的值为 x 的新结点，将 $*s$ 插入到 $*p$ 的前面。与后插不同的是：首先要找到 $*p$ 的前驱 $*q$ ，然后再完成在 $*q$ 之后插入 $*s$ ，设单链表头指针为 L ，操作如下：

```
q=L;  
while (q->next!=p)  
    q=q->next;    /*找*p的直接前驱*/  
s->next=q->next;  
q->next=s;
```



(3) 插入运算 $\text{Insert_LinkList}(L, i, x)$

算法思路:

- ① 找到第 $i-1$ 个结点；若存在继续2，否则结束；
- ② 申请、填装新结点；
- ③ 将新结点插入，结束。

```

) Status ListInsert(LinkList &L, int i, ElemType e) {
    //在带头结点的单链表L中第i个位置插入值为e的新结点
    LNode *p = L;
    int j = 0;
    while (p && (j < i - 1)) { //查找第i-1个结点, p指向该结点
        p = p->next;
        ++j;
    }
    if (!p || j > i - 1) return ERROR; //第i-1个结点不存在
    LNode *s = new LNode; //生成新结点
    s->data = e;
    s->next = p->next; //新结点的next指向第i-1个结点的next
    p->next = s; //第i-1个结点的next指向s
    return OK;
}

```

5.删除操作

(1) 删除结点:

设 p 指向单链表中某结点, 删除 $*p$ 。要实现对结点 $*p$ 的删除, 首先要找到 $*p$ 的前驱结点 $*q$, 然后完成指针的操作即可。指针的操作由下列语句实现:

```
q->next=p->next;
```

```
free(p);
```

显然找 $*p$ 前驱的时间复杂度为 $O(n)$ 。

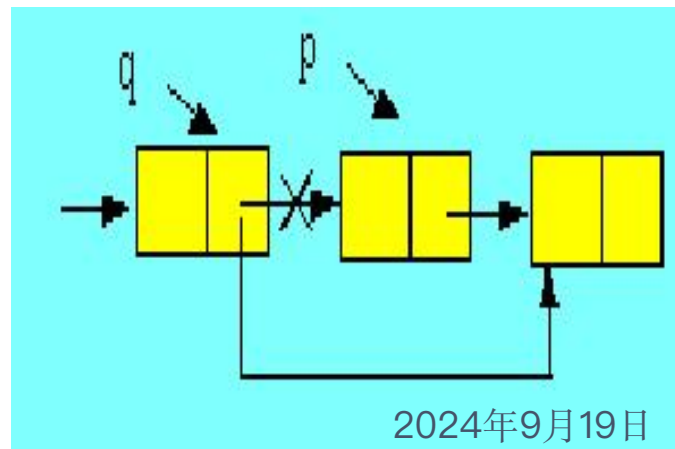
若要删除 $*p$ 的后继结点(假设存在), 则可以直接完成:

```
s=p->next;
```

```
p->next=s->next;
```

```
free(s);
```

该操作的时间复杂度为 $O(1)$ 。



(2) 删除运算 Del_LinkList(L,i)

算法思路:

- ①找到第 $i-1$ 个结点；若存在继续2，否则结束；
- ②若存在第 i 个结点则继续3，否则结束；
- ③删除第 i 个结点，结束。

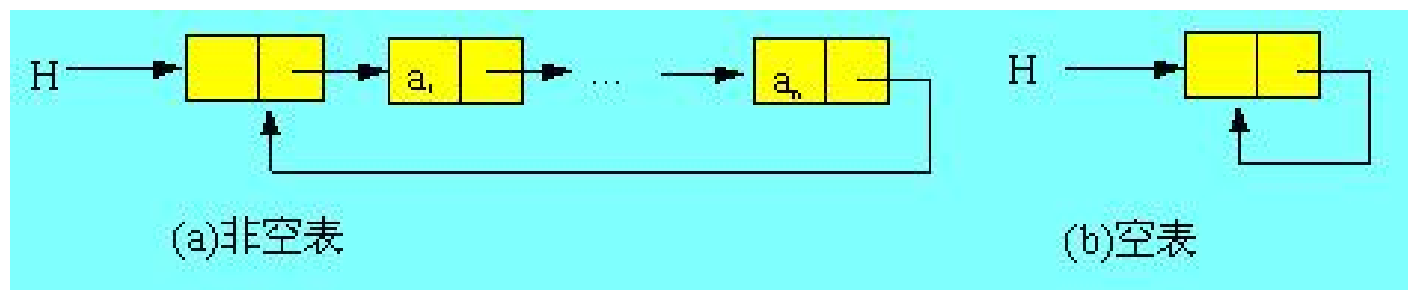
```

} Status ListDelete(LinkList &L, int i) {
    //在带头结点的单链表L中，删除第i个元素
    LNode *p = L;
    int j = 0;
    while (p && (j < i - 1)) { //查找第i-1个结点，p指向该结点
        p = p->next;
        ++j;
    }
    if (!p || j > i - 1) return ERROR; //第i-1个结点不存在
    LNode *q = p->next; //临床保存被删结点的地址以备释放
    p->next = q->next; //删除结点的前驱结点指向删除结点的后继结点
    delete q; //释放删除结点的空间
    return OK;
}

```

2.3.3 循环链表

对于单链表而言，最后一个结点的指针域是空指针，如果将该链表头指针置入该指针域，则使得链表头尾结点相连，就构成了单循环链表。



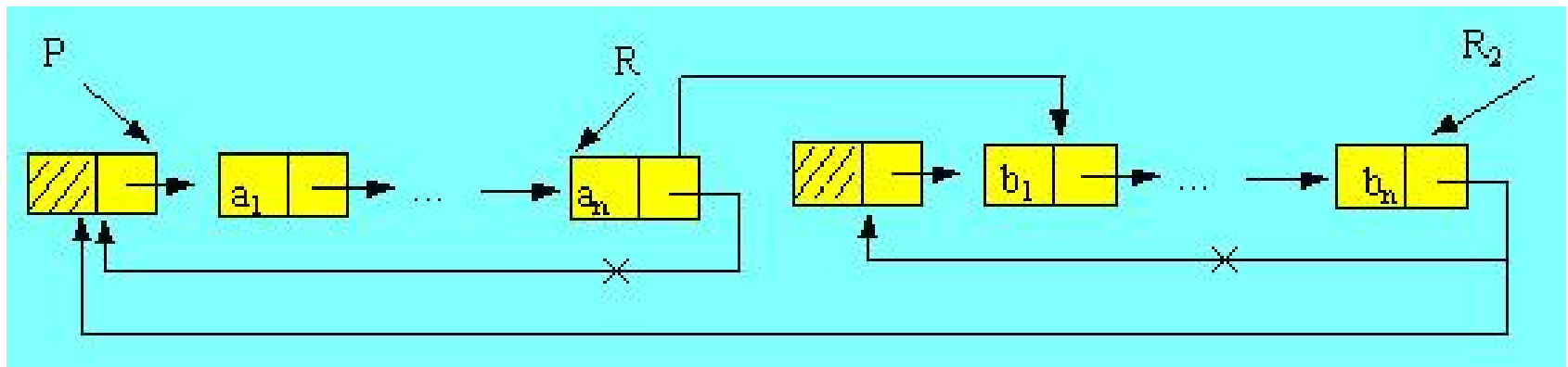
例：对两个单循环链表 H_1 、 H_2 的连接操作，是将 H_2 的第一个数据结点接到 H_1 的尾结点，如用头指针标识，则需要找到第一个链表的尾结点，其时间复杂性为 $O(n)$ ，而链表若用尾指针 R 、 R_2 来标识，则时间性能为 $O(1)$ 。操作如下：

```
P = R->next;    /*保存第一个表的头结点指针*/
```

```
R->next = R2->next->next;    /*头尾连接*/
```

```
free(R2->next);    /*释放第二个表的头结点*/
```

```
R2->next = P;    /*组成循环链表*/
```



2.3.4 双向链表

单链表的结点中只有一个指向其后继结点的指针域 next，找后继的时间性能是 $O(1)$ ，找前驱的时间性能是 $O(n)$ ；可以付出空间的代价使得找前驱的时间性达到 $O(1)$ ：每个结点再加一个指向前驱的指针域。

用这种结点组成的链表称为双向链表。

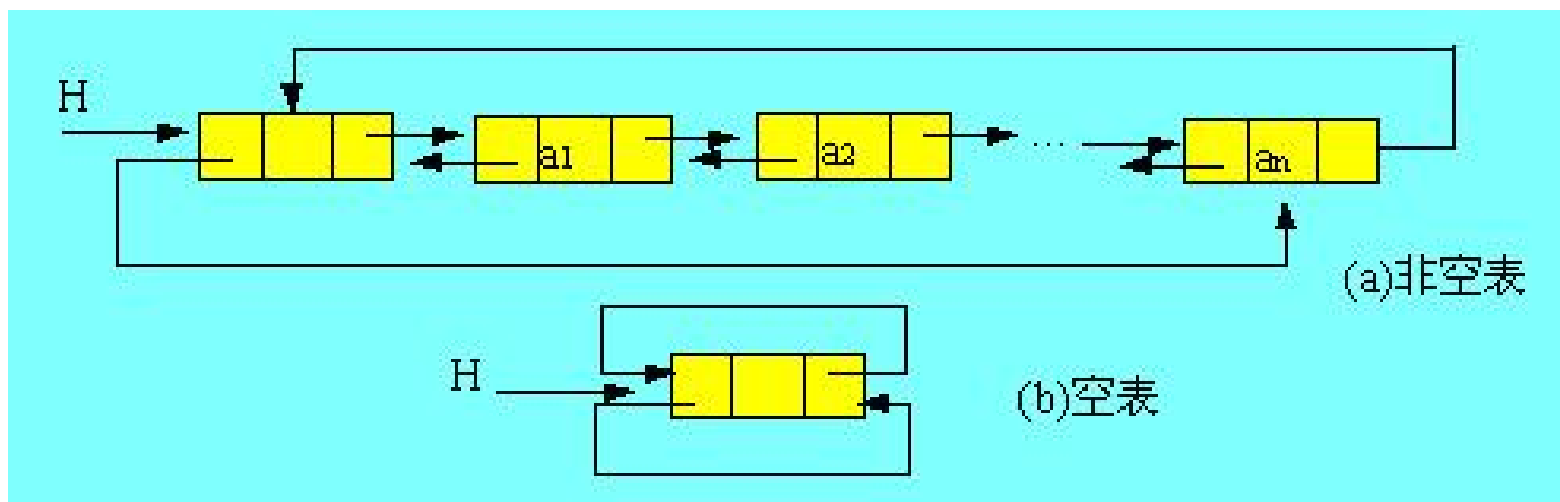
结点形式如下：



(1) 双向链表的结点定义如下：

```
typedef struct dlnode  
{ElemType data;  
    struct dlnode *prior,*next;  
}DLNode,*DLinkList;
```

(2) 和单链表类似，双向链表通常也是用头指针标识，也可以带头结点和做成循环结构。



(3) 在双向链表中，通过某结点的指针p既可以直接得到它的后继结点的指针p->next，也可以直接得到它的前驱结点的指针p->prior。

$$p \rightarrow \text{prior} \rightarrow \text{next} == p == p \rightarrow \text{next} \rightarrow \text{prior}$$

(4) 在双向链表中插入一个结点：

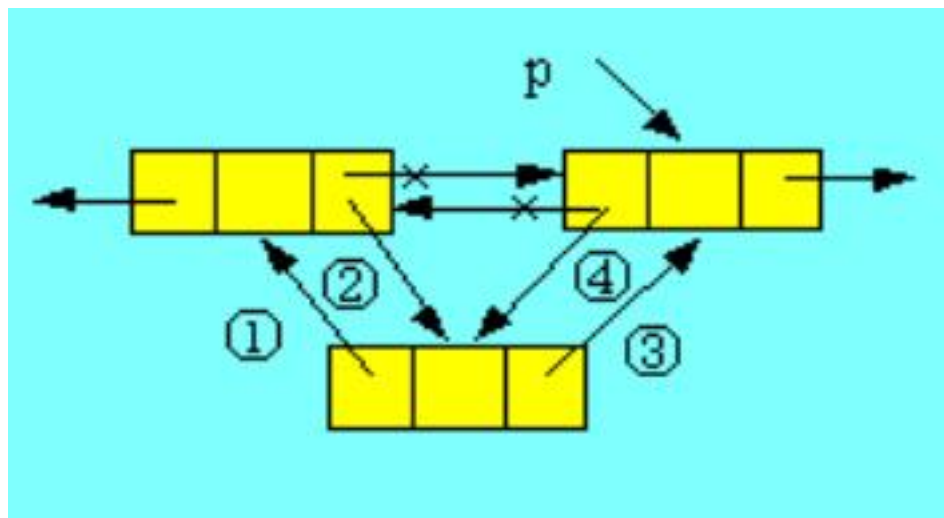
设 p 指向双向链表中某结点， s 指向待插入的值为 x 的新结点，将 $*s$ 插入到 $*p$ 的前面。操作如下：

① $s \rightarrow \text{prior} = p \rightarrow \text{prior};$

② $p \rightarrow \text{prior} \rightarrow \text{next} = s;$

③ $s \rightarrow \text{next} = p;$

④ $p \rightarrow \text{prior} = s;$



上面指针操作的顺序不是唯一的，但也不是任意的，操作①必须要放到操作④的前面完成，否则 $*p$ 的前驱结点的指针就丢掉了。

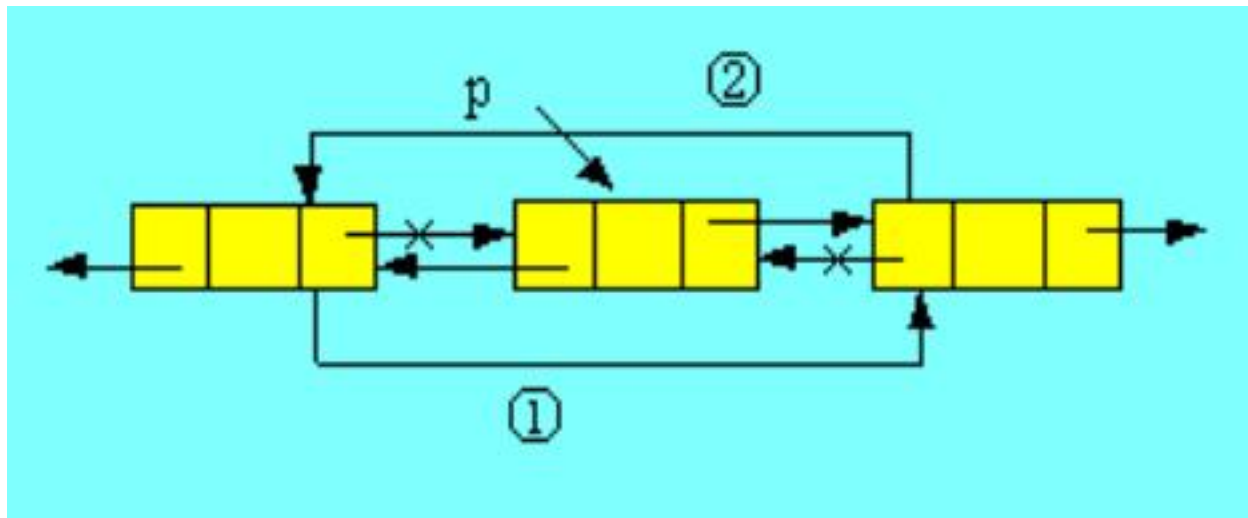
(5) 在双向链表中删除指定结点：

设 p 指向双向链表中某结点，删除 $*p$ 。

操作如下：① $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$;

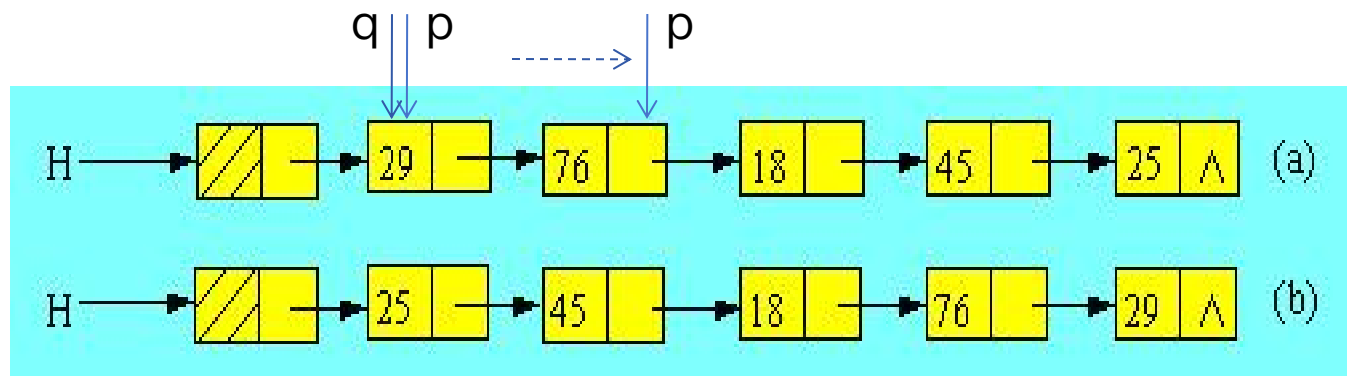
② $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$;

③ $\text{free}(p)$;



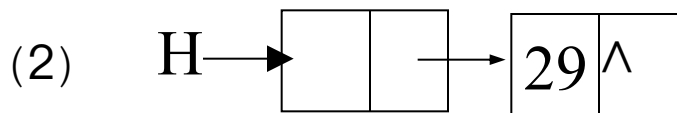
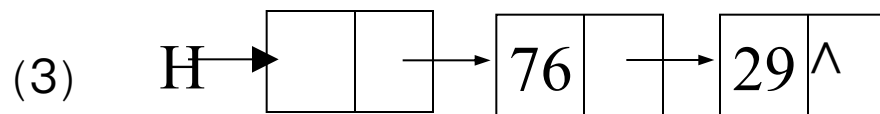
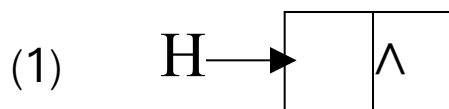
2.3.6 单链表应用举例

【例】已知单链表H如下图所示，写一算法将其逆置。

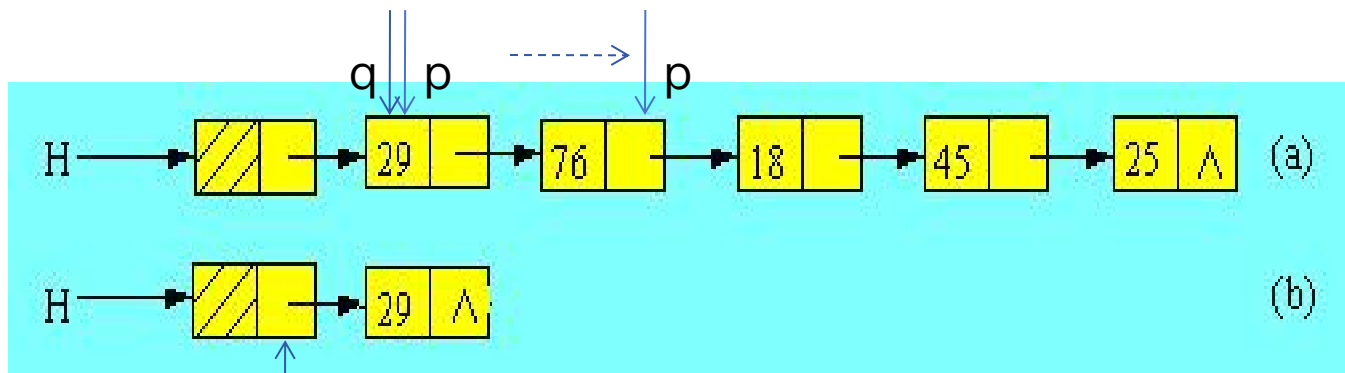


算法思路:

依次取原链表中的每个结点，将其作为第一个结点插入到新链表中去，指针q用来当前要处理的结点，指针p指向下一个要处理结点，p为空时结束。



.....



```
void reverse(LinkList H)
```

```
{
    LNode *p;
    p = H->next;    /* p指向第一个数据结点*/
    H->next = NULL; /* 将原链表置为空表H*/
    while(p)
    {
        q = p; // q指向要处理的这个结点
        p = p->next; // p移动到下一个要处理的结点
        q->next = H->next; // 将当前q结点插到新链表头结点的后面
        H->next = q;
    }
}
```

该算法只是对链表中顺序扫描一遍即完成了逆置，所以时间性能为 $O(n)$

【例】设有两个单链表A、B，其中元素**递增**有序，编写算法将A、B归并成一个按元素值**递减**（允许有相同值）有序的链表C，要求用A、B中的原结点形成，不能重新申请结点。

算法思路：

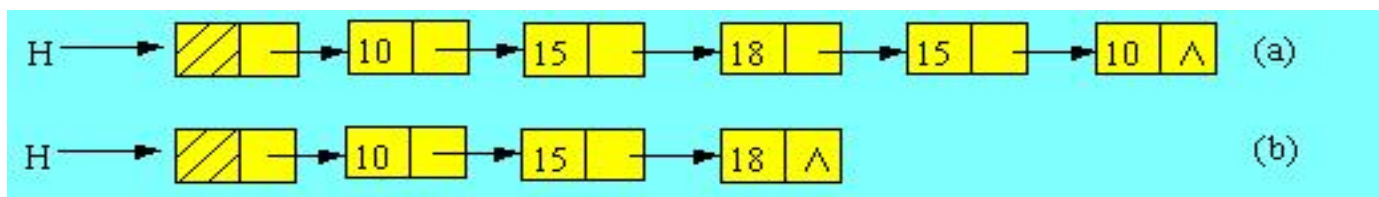
利用A、B两表有序的特点，依次进行比较，**将当前值较小者摘下，插入到C表的头部**，得到的C表则为递减有序的。

```

LinkedList merge(LinkedList A, LinkedList B){
    //设A、B均为带头结点的单链表
    LinkedList C; LNode *p, *q, *s;
    p = A->next; q = B->next;
    C = A; //合并后的链表使用A的头结点
    C->next = NULL;
    free(B);
    while(p&&q){
        if(p->data < q->data){
            s = p; p = p->next;
        }else{
            s = q; q = q->next; //s指向原AB链表中较小的结点
        }
        s->next = C->next; //把较小的结点插入到C表的头部
        C->next = s;
    }
    if(p==NULL) p=q; // p指向剩余链表中的第一个结点
    while(p){ //把剩余的结点一个个摘下，插入到C表的头部
        s = p; p = p->next;
        s->next = C->next;
        C->next = s;
    }
}

```

【例】 已知单链表H如下图所示，写一算法，删除其重复结点，即实现如下图所示的操作。



算法思路：

用指针p指向第一个数据元素结点，从它的后继结点开始到表的结束，查找与其值相同的结点并删除之；p指向下一个结点；依此类推，p指向最后结点时算法结束。

单链表中删除重复结点代码编写流程

```
void pur_LinkList(LinkList H)
{
    // 外层循环，用指针p依次指向链表中的每个结点
    while()
    {
        //内层循环，遍历p后面的结点，看是否有和p指向的
        结点的值相同的，有的话就删除
        while( )
        {
        }
    }
}
```

单链表中删除重复结点代码编写流程

```
void pur_LinkList(LinkList H)
{
    // 外层循环，用指针p依次执行链表中的每个结点
    while(p)
    {
        //内层循环，遍历p后面的结点，看是否有和p指向的
        结点的值相同的，有的话就删除
        while( )
        {
        }
        p= p->next;
    }
}
```

单链表中删除重复结点代码编写流程

```
void pur_LinkList(LinkList H)
{
    LNode *p;
    p = H->next;
    // 外层循环，用指针p依次执行链表中的每个结点
    while(p)
    {
        //内层循环，遍历p后面的结点，看是否有和p指向的结点的值相同的，有的话就删除
        while( )
        {
        }
        p= p->next;
    }
}
```

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {
```

//内层循环，遍历p后面的结点，看是否有和p指向的结点的值相同的，有的话就删除

```
        q = p->next; //q指向p的后继结点
```

```
        while(q)
```

```
        {
```

....//判断q执行的结点值是否和p指向

```
            q=q->next;
```

```
        }
```

```
        p= p->next;
```

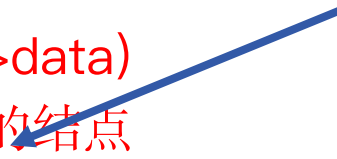
```
    }
```

```
}
```

```
void pur_LinkList(LinkList H)
```

```
{  
    LNode *p,*q;  
    p = H->next;  
    while(p)  
    {  
        q = p->next; //q指向p的后继结点  
        while(q)  
        {  
            if(q->data==p->data)  
            { //删除q指向的结点  
            }  
            q=q->next;  
        }  
        p= p->next;  
    }  
}
```

需要让q的前驱指向q的后继



```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q, *q_pre;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {
```

```
        q_pre = p;
```

```
        q = p->next; //q指向p的后继结点
```

```
        while(q)
```

```
        {
```

```
            if(q->data==p->data)
```

```
            {}
```

```
            q_pre = q_pre->next; //或者是q_pre=q;
```

```
            q=q->next;
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q, *q_pre;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q = p->next; //q指向p的后继结点
```

```
        q_pre = p;
```

```
        while(q)
```

```
        {    if(q->data==p->data)
```

```
            {q_pre->next=q->next;
```

```
              free(q)
```

```
            }
```

```
            q_pre = q_pre->next; //或者是q_pre=q;
```

```
            q=q->next;
```

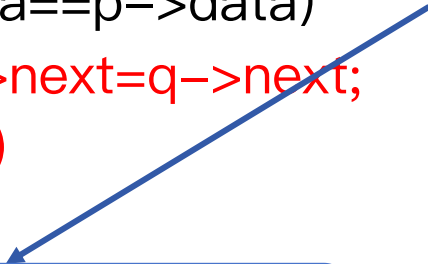
```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

删除结点q时这两句就有问题了



```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q, *q_pre;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q = p->next; //q指向p的后继结点
```

```
        q_pre = p;
```

```
        while(q)
```

```
        {    if(q->data==p->data)
```

```
            {q_pre->next=q->next;
```

```
                free(q);//删除q指向的结点, q还需要指向它的下一
```

个结点

```
            }else{
```

```
                q_pre = q_pre->next; //或者是q_pre=q;
```

```
                q=q->next;}
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q, *q_pre, *r;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q = p->next; //q指向p的后继结点
```

```
        q_pre = p;
```

```
        while(q)
```

```
        {    if(q->data==p->data)
```

```
            {q_pre->next=q->next;
```

```
                r = q; q=q->nxet;
```

```
                free(r);//删除q指向的结点， q还需要指向它的下一
```

个结点

```
            }else{
```

```
                q_pre = q_pre->next; //或者是q_pre=q;
```

```
                q=q->next;}
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

q和q_pre密切相关，可以用一个变量表示另外一个变量，对代码进行优化

q= q_pre->next

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p,*q, *q_pre, *r;
```

```
    p = H->next;
```

```
    while(p)
```

```
    { q = p->next; //q指向p的后继结点
```

```
        q_pre = p;
```

```
        while(q q_pre->next)
```

```
        { if(q->data==p->data) (q_pre->next->data == p->data)
```

```
            {q_pre->next=q->next q->pre->next->next;
```

```
              r = q q_pre->next; q=q->next;
```

```
              free(r);//
```

```
            }else{
```

```
            q_pre = q_pre->next; //或者是q_pre=q;
```

```
            q=q->next;}
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

q和q_pre密切相关，可以用一个变量表示另外一个变量，对代码进行优化

q= q_pre->next

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p, *q_pre, *r;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q_pre = p;
```

```
        while(q_pre->next)
```

```
        {    if(q_pre->next->data == p->data)
```

```
            {q_pre->next= q->pre->next->next;
```

```
              r = q_pre->next;
```

```
              free(r);//
```

```
            }else{
```

```
            q_pre = q_pre->next; //或者是q_pre=q;
```

```
            }
```

```
        }
```

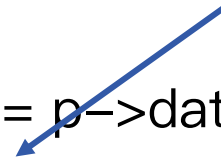
```
        p= p->next;
```

```
    }
```

```
}
```

整理代码

需要调换一下位置



```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p, *q_pre, *r;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q_pre = p;
```

```
        while(q_pre->next)
```

```
        {    if(q_pre->next->data == p->data)
```

```
            {r = q_pre->next;
```

```
              q_pre->next= r->next;
```

```
              free(r);//
```

```
            }else{
```

```
              q_pre = q_pre->next; //或者是q_pre=q;
```

```
            }
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

整理代码

把变量名q_pre换个名字,

改为q

```
void pur_LinkList(LinkList H)
```

```
{  
    LNode *p, *q, *r;  
    p = H->next;  
    while(p)  
    {  
        q = p;  
        while(q->next)  
        {  
            if(q->next->data == p->data)  
            {  
                r = q->next;  
                q->next = r->next;  
                free(r);  
            }  
            else{  
                q = q->next; //  
            }  
        }  
        p = p->next;  
    }  
}
```

整理代码

把变量名q_pre换个名字,
改为q

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p, *q, *r;
```

```
    p = H->next;
```

```
    while(p)
```

```
    {    q = p;
```

```
        while(q->next)
```

```
        {    if(q->next->data == p->data)
```

```
            {r = q->next;
```

```
                q->next= r->next;
```

```
                free(r);//
```

```
            }else{
```

```
                q = q->next; //
```

```
            }
```

```
        }
```

```
        p= p->next;
```

```
    }
```

```
}
```

其实不需要对最后一个元素去重，只需要保证倒数第二个元素时能进入while循环即可

把while(p)改为while(p->next)即可

while(p)，只要p不为空就可以进入循环，因此最后一个结点可以进入循环

while(p->next)，只要p的下一个结点不为空就可以进入循环，因此p指向倒数第二个元素时可以进入循环，p指向最后一个元素时不能进入循环

```
void pur_LinkList(LinkList H)
```

```
{
```

```
    LNode *p, *q, *r;
```

```
    p = H->next;
```

```
    while(p->next)
```

p->next是否一定合法呢?

```
{        q = p;
```

```
    while(q->next)
```

```
    {    if(q->next->data == p->data)
```

```
        {r = q->next;
```

```
            q->next= r->next;
```

```
            free(r);//
```

```
        }else{
```

```
            q = q->next; //
```

```
        }
```

```
    }
```

```
    p= p->next;
```

```
}
```

```
}
```

```
void pur_LinkList(LinkList H)
```

```
{  
    LNode *p, *q, *r;  
    p = H->next;  
    if(p==null) return;  
    while(p->next)  
    {  
        q = p;  
        while(q->next)  
        {  
            if(q->next->data == p->data)  
            {  
                r = q->next;  
                q->next = r->next;  
                free(r);  
            }else{  
                q = q->next; //  
            }  
        }  
        p = p->next;  
    }  
}
```

```

void pur_LinkList(LinkList H)
{
    LNode *p, *q, *r;
    p = H->next;    //p指向第一个结点
    if(p==NULL) return;
    while(p->next){
        q = p; //通过q->next来遍历p后续的结点
        while(q->next){ //从p的后继开始找重复结点
            if(q->next->data == p->data){//如果q->next为重复结点
                r = q->next; //r指向重复结点
                q->next = r->next;
                free(r)
            }else{
                q = q->next;
            }
        } //while(q->next)
        p = p->next;
    } //while(p->next)
}

```

单链表中删除重复结点，该算法的时间性能为 $O(n^2)$ 。

2024年9月19日

2.4 顺序表和链表的比较

顺序存储有3个优点：

- (1) 方法简单，各种高级语言中都有数组，容易实现。
- (2) 不用为表示结点间的逻辑关系而增加额外的存储开销。
- (3) 顺序表具有按元素序号随机访问的特点。

● 顺序存储两个缺点：

- (1) 在顺序表中做插入删除操作时，平均移动大约表中一半的元素，因此对 n 较大的顺序表效率低。
- (2) 需要预先分配足够大的存储空间，估计过大，可能会导致顺序表后部大量闲置；预先分配过小，又会造成溢出。

● 链表的优缺点恰好与顺序表相反。

在实际中怎样选取存储结构呢？通常有以下几点考虑：

1. 基于存储的考虑
2. 基于运算的考虑
3. 基于环境的考虑

2.5 单链表的应用：多项式及其运算

数学表示

$$p_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$$

$$[\text{例}] \quad pa(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$pb(x) = 8x + 22x^7 - 9x^8$$

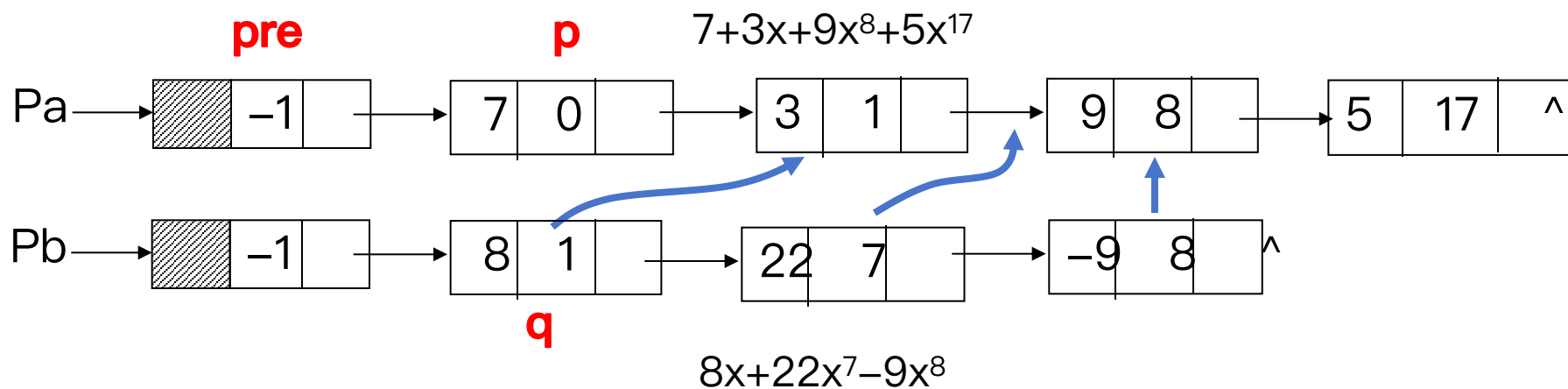
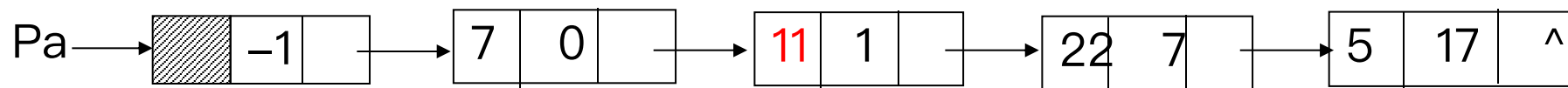
确定逻辑结构并且选择存储结构

线性表：数据元素 $\begin{cases} (p) & \text{仅保存系数，稀疏多项式时浪费空间} \\ (p, e) & \checkmark \end{cases}$

存储结构： $\begin{cases} \text{顺序表} & \text{不利于运算（需较多插入删除）} \\ \text{带头结点的单链表} & \checkmark \end{cases}$

存储结构定义

```
typedef struct {  
    float    coef;    // 系数  
    int      expn;    // 指数  
}ElemType;  
Typedef LinkList polynomial;
```



算法
实现
设多
项式
按指
数升
序排
列

```
void AddPolyn(polynomial &Pa, polynomial &Pb)
{
    CreatPolyn(Pa, m); CreatPolyn(Pb, n);
    p=Pa->next; q=Pb->next; pre=Pa;
    while (p && q){
        switch (compare(p->data.expn, q->data.expn)){
            case -1: //PA当前结点指数小
                pre->next = p; pre = p; p = p->next;
                break;
            case 0: //PA PB当前结点指数相等
                sum = p->data.coef+q->data.coef
                if(fabs(sum) > 1e-6){// sum不等于0
                    p->data.coef = sum;
                    pre->next = p; pre = p; p = p->next; //保留PA结点
                    temp = q; q = q->next; free(temp); //删除PB结点
                }else{
                    temp = p; p = p->next; free(temp); //删除PA结点
                    temp = q; q = q->next; free(temp); //删除PB结点
                }
                break;
            case 1: //PB当前结点指数小
                pre->next = q; pre = q; q = q->next;
                break;
        }
    }
    if(q)pre->next=q else pre->next=q; //把剩余结点保留下来
    free(Pb);
    OutputPolyn(Pa)
} //AddPolyn
```

```
void CreatPolyn(polynomial &polyn, int num)
{
    polyn=(polynomial)malloc(sizeof(LNode));
    polyn->data.expn=-1; polyn->next=NULL;
    pre=polyn;
    for (count=0; count<num; count++) {
        scanf(c,n);          //读入系数和指数
        p= (LNode *)malloc(sizeof(LNode));
        p->data.coef=c; p->data.expn=n;
        p->next=NULL;
        pre->next=p;
        pre=p;
    }
} //CreatPolyn
```



2.6 本章知识点小结

- 线性表的概念
 - 线性表的顺序存储
 - 顺序存储线性表的操作
 - 线性表的链式存储
 - 链式存储线性表的操作
 - 头结点及其作用
 - 单链表
 - 循环链表
 - 双向链表
 - 顺序存储线性表的特点
 - 链式存储线性表的特点
 - 单链表的应用：多项式及其运算
- 理解指针和引用的含义
 - 警惕指针丢失和内存泄漏
 - 留意边界条件和特殊情况
 - 举例画图，辅助思考
 - 多写多练，没有捷径